

UNIVERSITY OF INFORMATION TECHNOLOGY
VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY
FACULTY OF COMPUTER SCIENCE



FINAL PROJECT REPORT

COURSE: CS338.Q22 - PATTERN RECOGNITION

**SPIKEGPT AS AN AGENTIC AI:
EVALUATING SPIKING NEURAL NETWORKS
FOR TOOL CALLING IN COMMERCIAL
MANAGEMENT SYSTEMS**

Instructors: Duong Viet Hang, Tran Doan Thuyen
Team: Group 5
Reference paper: SpikeGPT: Generative Pre-trained Language Model
with Spiking Neural Networks
GitHub repository: <https://github.com/PhamQuocNam/CS338.git>
Implementation period: 05/2026–06/2026

No.	Student	Student ID
1	Pham Quoc Nam	23520984
2	Nguyen Khang Hy	23520662


SPIKING NEURAL NETWORKS

Ho Chi Minh City, June 2026

Contents

ABSTRACT	4
1 Introduction	5
1.1 Research Objectives	5
1.2 Summary of Findings	5
1.3 Project Scope and Contributions	6
2 Related Work	7
2.1 Agentic AI and Tool Calling	7
2.2 Spiking Neural Networks and SpikeGPT	7
2.3 Fine-Tuning Small Language Models as Baselines	7
2.4 Self-Training for Tool Calling Data Generation	8
2.5 Position of This Project	8
3 Methods	9
3.1 System Architecture and End-to-End Pipeline	9
3.1.1 Component 1 — User Query	9
3.1.2 Component 2 — SpikeGPT Agent	9
3.1.3 Component 3 — Execution Engine	10
3.1.4 Component 4 — Supabase Cloud Database	10
3.1.5 Component 5 — Observation	10
3.2 Tool Specification	11
3.2.1 Group 1 — Query Tools (Read-Only)	11
3.2.2 Group 2 — Mutation Tools (Write)	12
3.2.3 Tool Routing Complexity	12
3.3 Model Architectures	13
3.3.1 SpikeGPT (SNN-based)	13
3.3.2 HeadQK for Copy Support	13
3.3.3 Training Objective	14
3.3.4 Implementation and Training Details	15
3.3.5 GPT-2 Small — Zero-Shot Baseline (124M)	16
3.3.6 Fine-Tuned GPT-2 Medium — SFT Reference (355M)	16

3.4	Dataset Construction and Preprocessing	16
3.4.1	Construction Pipeline	16
3.4.2	Dataset Scale and Splits	17
3.4.3	Quality Assurance	18
3.4.4	Preprocessing and Data Augmentation	19
3.4.5	Binary Spike Encoding for SpikeGPT	19
3.5	Data Pipeline Summary	19
3.6	Dataset Validity Analysis	20
3.6.1	Tool and Intent Distribution	20
4	Experiments	22
4.1	Experimental Setup	22
4.2	Evaluation Metrics	22
4.3	Main Results	23
4.4	Analysis and Failure Modes	23
4.4.1	JSON Validity and Intent Accuracy	23
4.4.2	Args Exact Match — The Critical Bottleneck	24
4.4.3	Hallucination in GPT-2 Small	24
4.4.4	SpikeGPT Ablation Insights	24
4.5	Additional Ablation Studies	24
4.5.1	Copy-Aware Loss	24
4.5.2	ID Format Stress Test	25
4.5.3	Why Valid JSON Is Not Enough	25
5	Discussion	26
5.1	Viability of SNNs for Agentic AI	26
5.2	The Encoding-Precision Trade-off	26
5.3	Deployment Considerations	26
5.4	Argument Extraction as a Copy-Binding Problem	26
5.5	Lessons from Failed Improvements	27
5.6	Practical System Design	27
5.7	Future Improvements	27
6	Conclusion	29

ABSTRACT

Modern Agentic AI systems predominantly rely on Transformer-based architectures, which demand substantial computational resources and incur high operational costs. This raises a pressing research question: *Can Spiking Neural Network (SNN) architectures, with their superior energy efficiency, be applied to execute complex Agentic tasks—such as Tool Calling—in commercial management scenarios?* This paper investigates SpikeGPT, a hybrid architecture combining SNNs with modern language-processing mechanisms, as the backbone of an intermediate Agentic Chatbot. The system autonomously routes requests and extracts parameters from a database using natural language. To benchmark SpikeGPT, we construct a domain-specific JSON dataset and evaluate two Transformer baselines: GPT-2 Small and fine-tuned GPT-2 Medium. Experiments show that SpikeGPT Finetune (epoch 220) achieves a valid JSON generation rate of 99.80% and an Intent Accuracy of 94.10%, confirming its structural understanding of Tool Calling workflows. However, for tasks requiring absolute precision—specifically argument extraction (Args Exact Match)—SpikeGPT reaches only 13.94%, significantly below the $\approx 68\%$ achieved by GPT-2 variants. We trace this gap to the spike-encoding mechanism, which impedes gradient convergence in fine-grained token-level tasks.

Keywords: Spiking Neural Networks, Agentic AI, Tool Calling, SpikeGPT, Large Language Models, Function Calling, Commercial Management.

Chapter 1

Introduction

The rapid advancement of Large Language Models (LLMs) has made it feasible to build intelligent assistants capable of interacting directly with databases through *Tool Calling* mechanisms [3]. Instead of writing complex SQL or GraphQL queries, users can express requests in natural language, and the system automatically analyzes intent and retrieves the required data. Modern Agentic systems almost exclusively rely on Transformer-based architectures, causing enormous resource consumption and limiting deployment on edge hardware or in latency-sensitive environments. This motivates the search for an alternative architecture that is energy-optimal while preserving language reasoning capability.

1.1 Research Objectives

This project investigates SpikeGPT [1]—a representative of Spiking Neural Networks (SNNs)—to answer a core question:

Can SpikeGPT function as an AI Agent, and to what extent can it effectively execute multi-step Tool Calling?

Rather than limiting evaluation to standard text generation, we deploy SpikeGPT in a rigorous commercial management environment. The model must:

1. Parse natural language semantics;
2. Reason and select the appropriate tool (*Tool Routing*);
3. Generate structured API calls in JSON format.

For comparison, we establish baselines using two Decoder-only Transformer models: GPT-2 Small (124M parameters) and fine-tuned GPT-2 Medium (355M parameters).

1.2 Summary of Findings

Our experiments yield three key findings:

- **Feasibility:** SpikeGPT can learn multi-turn dialogue structure and agent-style reasoning. The best fine-tuned variant (epoch 220) achieves 99.80% valid JSON rate and 94.10% Intent Accuracy.
- **Challenge:** Args Exact Match remains a critical bottleneck. While Transformer baselines reach $\approx 68.30\%$, SpikeGPT peaks at 13.94%.
- **Conclusion:** SpikeGPT shows great promise for resource-constrained settings, but

requires advances in data encoding to compete with Transformers on precision-critical Agentic tasks.

1.3 Project Scope and Contributions

This report studies SpikeGPT as a practical tool-calling agent rather than as a general-purpose chatbot. The system is placed in a commercial management setting, where a user may ask to check product inventory, create an order, update an order, delete an order, retrieve an order, or analyze revenue over a date range. The model must not only produce fluent text; it must produce a structured tool call that can be executed by a backend service.

Compared with the original SpikeGPT paper, this project changes the evaluation setting in three important ways. First, the target output is a JSON function call, which is much stricter than open-ended language generation. Second, the task is evaluated at several levels: JSON syntax, tool selection, and exact argument copying. Third, the project compares SpikeGPT with GPT-2 baselines to make the limitation of the spiking architecture clearer in a familiar language-modeling setting.

The main contributions of the project are as follows:

1. We build a Vietnamese and English-style commercial management tool-calling dataset with six tools and typed JSON arguments.
2. We fine-tune SpikeGPT-216M and evaluate several SpikeGPT variants, including a NoHeadQK setting, from-scratch training, and fine-tuning from a pretrained checkpoint.
3. We test a lightweight HeadQK mechanism to improve argument copying, and we analyze why it helps but does not close the gap with Transformer attention.
4. We evaluate a copy-aware loss as an ablation study, showing that weighting copy-sensitive tokens can improve Args Exact Match but may reduce intent classification quality.
5. We separate model quality into practical metrics so that a high JSON score is not mistaken for a fully usable tool-calling agent.

The project therefore does not simply reproduce the SpikeGPT paper. It uses the paper as a starting point and then tests whether the architecture still works when the output must be machine-executable and exact.

Chapter 2

Related Work

2.1 Agentic AI and Tool Calling

With the maturation of LLMs and Agentic AI architectures, virtual assistants capable of directly interfacing with database systems via tool invocation have become practical. Modern systems typically bypass hand-written SQL by automatically parsing user intent and routing to appropriate APIs. The dominant paradigm relies on large-scale Transformer architectures, motivating research into more resource-efficient alternatives.

2.2 Spiking Neural Networks and SpikeGPT

To address the energy costs of Transformers, Spiking Neural Networks (SNNs) have emerged as a promising direction [4]. SNNs communicate via sparse binary spike events rather than continuous floating-point activations, leading to significant reductions in multiply-accumulate (MAC) operations.

SpikeGPT [1], introduced by Zhu *et al.* in 2023, is the first generative language model fully trained with SNNs at the scale of tens to hundreds of millions of parameters. It is built on the RWKV (Receptance Weighted Key Value) architecture, replacing the quadratic-complexity $O(N^2)$ self-attention of standard Transformers with a linear-recurrent formulation at $O(N)$ complexity—naturally compatible with the temporal event-driven nature of SNNs. Key characteristics of SpikeGPT include:

- **Binary spike activations**, which drastically reduce computational operations;
- **Event-driven computation**, activating neurons only when significant signals are present;
- **Linear sequence complexity**, suitable for long-context language tasks;
- Models trained at $\approx 45\text{M}$ and $\approx 216\text{M}$ parameters; reported to require up to $20\times$ fewer operations on neuromorphic hardware.

Limitations include difficulty in training due to non-differentiable spike activation functions, slower convergence, and a less mature tooling ecosystem compared to PyTorch/TensorFlow.

2.3 Fine-Tuning Small Language Models as Baselines

To objectively evaluate SpikeGPT’s Agentic performance, we use GPT-2 Small [2] as a zero-shot NLP baseline and fine-tuned GPT-2 Medium as a Supervised Fine-Tuning (SFT) reference. The latter is trained on a domain-specific Tool Calling dataset to establish a strong

upper bound for argument extraction accuracy within the small-model regime.

2.4 Self-Training for Tool Calling Data Generation

A core challenge in training Agentic systems is reliance on manually annotated tool-use datasets. [3] demonstrated that LLMs can automatically synthesize high-quality tool-use traces without requiring large human-annotated corpora. Inspired by this methodology, our project uses a powerful LLM (Gemini Pro) to expand a human-curated gold-standard dataset into thousands of diverse JSON conversation samples.

2.5 Position of This Project

The original SpikeGPT work mainly evaluates language modeling and classification ability through benchmarks such as Enwik8, WikiText, and common natural language understanding datasets. Those benchmarks are useful for showing that a spiking language model can scale, but they do not directly test whether the model can call external tools safely. In a tool-calling system, a sentence that is almost correct may still be unusable if one identifier, date, or quantity is wrong.

This project therefore uses SpikeGPT in a stricter setting. The model must learn a schema, choose one tool from a small API set, and copy the required arguments from the user request. This makes the task closer to a real agentic workflow. It also creates a natural stress test for RWKV-style recurrent compression and spike-based representations, because exact copying requires preserving surface-level details, not only the broad meaning of the prompt.

Transformer baselines are included not only to obtain high scores, but also to explain the type of error SpikeGPT makes. Attention gives GPT-style models a direct path from the current decoding position back to earlier tokens. SpikeGPT compresses the past through hidden states and spike events. The comparison between the two families helps us understand whether the main difficulty comes from the dataset, the training process, or the architecture itself.

Chapter 3

Methods

3.1 System Architecture and End-to-End Pipeline

We propose an Agentic Chatbot that acts as an intelligent intermediary for commercial management workflows. The system accepts natural language user queries, reasons over the appropriate API tool to invoke, parses structured arguments, executes the call against a cloud database, and synthesizes the raw result into a human-readable response. This design eliminates the requirement for managers to write SQL or GraphQL queries and reduces the risk of manual data-entry errors. The end-to-end pipeline comprises five sequential components, detailed below.

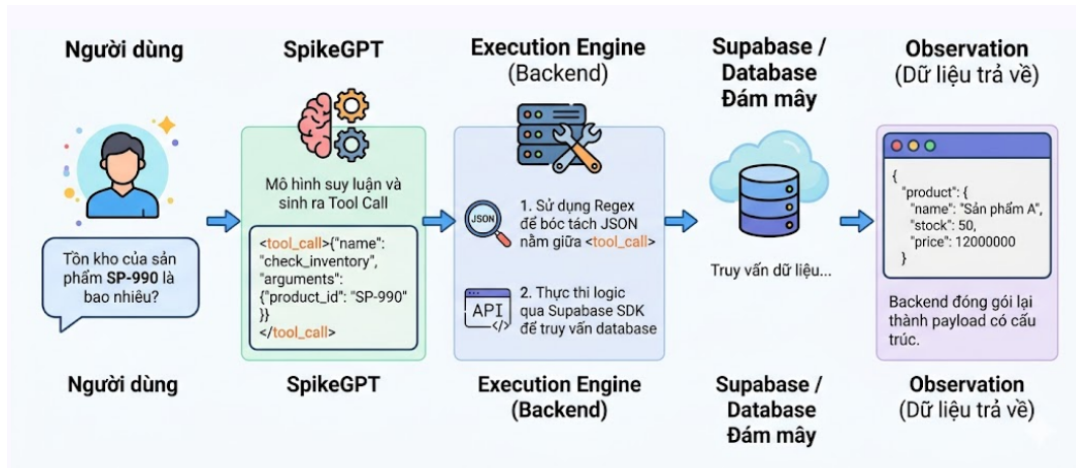


Figure 3.1: End-to-end pipeline of the proposed Agentic Chatbot system. Arrows denote the data flow from user query to structured observation.

3.1.1 Component 1 — User Query

The pipeline is initiated by a natural-language utterance from the manager. Queries range from simple single-entity lookups (e.g., “How many units of SP-990 are left in stock?”) to multi-parameter requests (e.g., “Update order ORD-112 to 5 units and change the address”). No structured syntax or domain-specific keywords are required; the system handles free-form Vietnamese and English input.

3.1.2 Component 2 — SpikeGPT Agent

The user query is fed directly into the SpikeGPT language model (or the Transformer baseline during comparative experiments). The model acts as the central *reasoning agent*: it parses

the semantic intent of the query, selects the appropriate tool from the six-tool vocabulary, and generates a structured Tool Call in a tagged format:

Listing 3.1: SpikeGPT Tool Call output (inventory query).

```
1 <tool_call>
2 {
3   "name": "check_inventory",
4   "arguments": {
5     "product_id": "SP-990"
6   }
7 }
8 </tool_call>
```

The model emits null as the content field of the assistant role whenever a tool call is triggered, signaling to the downstream Execution Engine that a function invocation—not a free-text reply—is expected.

3.1.3 Component 3 — Execution Engine

The Execution Engine is the middleware layer between the language model and the database. It performs two sequential operations:

1. **JSON Extraction via Regex:** A regular-expression parser extracts the structured JSON payload enclosed within `<tool_call> ... </tool_call>` delimiters, tolerating minor whitespace or formatting variations.
2. **API Execution via Supabase SDK:** The extracted JSON is validated against the target tool's parameter schema and dispatched through the Supabase client SDK (e.g., `check_inventory` $\mapsto$ a `SELECT` query on the inventory table filtered by `product_id`).

3.1.4 Component 4 — Supabase Cloud Database

The system's persistent data store is hosted on Supabase, a PostgreSQL-backed cloud platform providing real-time query capabilities, row-level security, and a RESTful SDK. All six tool categories—inventory records, order entries, revenue transactions, and product metadata—are stored as relational tables within a single Supabase project.

3.1.5 Component 5 — Observation

Upon successful query execution, the database returns a raw JSON payload, which is packaged into a structured Observation object and injected back into the conversation as a `role: tool` message:

Listing 3.2: Structured Observation returned from Supabase.

```

1 {
2   "product": {
3     "name": "Product A",
4     "stock": 50,
5     "price": 12000000
6   }
7 }

```

This Observation is passed back to the agent, which synthesizes it into a concise natural-language response (e.g., “*Product SP-990 currently has 50 units in stock, priced at 12,000,000 VND*”). The full five-component cycle completes in a single agentic turn.

Design Rationale: The tagged `<tool_call>` delimiter scheme was adopted because small language models (including SpikeGPT) occasionally introduce minor JSON formatting errors. Regex-based extraction is robust to such perturbations, decoupling model output quality from pipeline reliability.

3.2 Tool Specification

The Agentic Chatbot is equipped with six domain-specific tools organized into two functional groups. Table 3.1 summarizes the complete tool inventory.

Table 3.1: Complete tool inventory for the Agentic System.

Type	Tool Name	Parameters	Description
Q	check_inventory	product_id	Retrieves current stock quantity for a specific product ID.
Q	revenue_analysis	start_date, end_date	Computes total revenue statistics within a date range.
Q	get_order	order_id	Fetches all existing details of a specific customer order.
M	create_order	product_id, qty, addr	Places a new order into the database.
M	update_order	order_id, qty*, addr*	Modifies parameters of an existing order.
M	delete_order	order_id	Cancels or permanently removes an order.

^Q Query operations (read-only); safe to execute without side effects.

^M Mutation operations (write/update); modifies database state.

* Indicates an optional parameter. For update tools, only fields that need modification must be supplied.

3.2.1 Group 1 — Query Tools (Read-Only)

check_inventory. Retrieves the remaining stock count for a given product (product_id: string).

revenue_analysis. Computes total revenue and growth trends over a specified date range (start_date, end_date: ISO 8601 strings).

get_order. Fetches full details of an existing order (order_id: string).

3.2.2 Group 2 — Mutation Tools (Write)

create_order. Places a new order with three required fields: product_id (string), quantity (integer), and address (string).

update_order. Modifies one or more fields of an in-progress order. Only changed fields are required: order_id (required), quantity (optional integer), address (optional string).

delete_order. Cancels or permanently removes an order from the system (order_id: string).

3.2.3 Tool Routing Complexity

The six-tool design intentionally exercises both intent disambiguation (e.g., distinguishing get_order from update_order when partial order information is given) and argument completeness (e.g., create_order requires three parameters while check_inventory requires only one). This asymmetry makes the benchmark sensitive to both high-level reasoning and low-level entity extraction—directly probing the SpikeGPT vs. Transformer performance gap identified in Section 4.

3.3 Model Architectures

3.3.1 SpikeGPT (SNN-based)

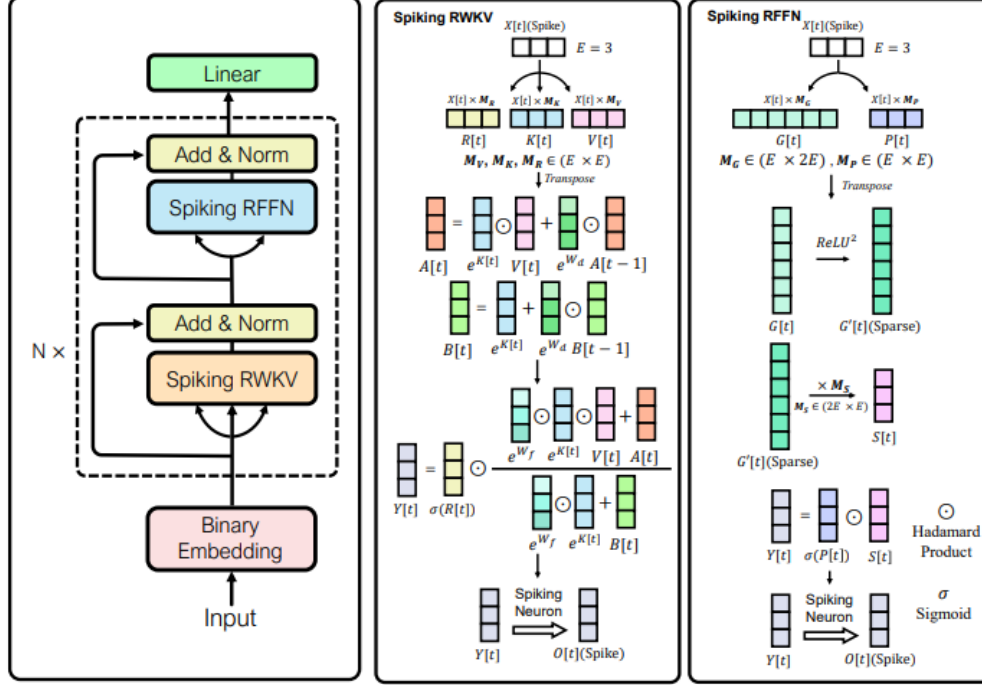


Figure 3.2: Architectural overview of the SNN-based SpikeGPT model.

SpikeGPT serves as the primary subject of investigation. Built on RWKV with binary spike activations, it processes token sequences event-driven and recurrently. We evaluate multiple ablation variants:

- **Scratch (ep. 78, 220):** Trained from scratch on our dataset.
- **NoHeadQK (ep. 78):** Ablation removing the HeadQK mechanism.
- **Finetune (ep. 220):** Fine-tuned from a pre-trained checkpoint.

3.3.2 HeadQK for Copy Support

HeadQK is the main architectural modification used in our SpikeGPT experiments. The motivation is simple: tool calling often requires the model to repeat an identifier from the user prompt, such as `product_id` or `order_id`. A recurrent model can learn the general intent, but it may not preserve the exact string well enough when producing the JSON arguments. HeadQK adds a lightweight attention-like signal near the output head so that the model has an additional way to compare the current decoding state with previous states.

Let $H = [h_1, \dots, h_T]$ be the sequence of hidden states from SpikeGPT. The added head projects hidden states into query and key vectors:

$$Q = HW_Q, \quad K = HW_K,$$

where $W_Q, W_K \in \mathbb{R}^{d \times d_q}$, and $d_q = 256$ in our experiments. A causal score matrix is then computed and normalized as:

$$S = \frac{QK^\top}{\sqrt{d_q}} + M_{\text{causal}},$$

$$A = \text{softmax}(S),$$

where M_{causal} prevents the model from attending to future tokens. In implementation, this extra signal is used to adjust the output distribution before token prediction. The goal is not to turn SpikeGPT into a full Transformer, but to give it a small copying aid for fields that must match the prompt exactly.

This change has a clear trade-off. It adds computation and weakens the “pure” lightweight motivation of SpikeGPT. However, for tool calling, exact argument copying is important enough that this cost is acceptable as an experimental modification. The ablation results in Chapter 4 show that HeadQK improves Args Exact Match from zero in the NoHeadQK setting, although it is still far below GPT-2.

3.3.3 Training Objective

All models are trained as causal language models. Given an input sequence $x_1, \dots, x_T$, the model learns to predict the next token:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

In the original training setting, this corresponds to standard token-level cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t \mid x_{<t}).$$

For the main model, we keep this standard objective. We also test a copy-aware weighted loss as an ablation. The idea is to assign a higher weight to target tokens that already appear in the previous context window, because these tokens are likely to be copied arguments:

$$\mathcal{L}_{\text{copy}} = -\frac{1}{\sum_t w_t} \sum_{t=1}^T w_t \log p_\theta(x_t \mid x_{<t}),$$

where

$$w_t = \begin{cases} \alpha, & x_t \in \{x_{t-L}, \dots, x_{t-1}\}, \\ 1, & \text{otherwise.} \end{cases}$$

In our experiment, $\alpha = 2.0$ and $L = 512$. This loss does not change the task: the model still predicts the next token. It only changes how much each token contributes to the gradient. As discussed later, this improves argument copying slightly but hurts intent and F1, so it is

reported as an ablation rather than the final selected model.

3.3.4 Implementation and Training Details

The SpikeGPT implementation is based on the open-source code released with the SpikeGPT paper. We keep the core RWKV-style recurrent blocks, the spike-based activation design, and the CUDA WKV operation. The project-specific changes are made around the training data, the output format, the HeadQK option, and the evaluation pipeline. This keeps the comparison fair: the backbone remains SpikeGPT, while the task is changed from general language modeling to structured tool calling.

Before training, each conversation is converted into a plain text sequence following the same role order used during evaluation. The assistant output contains the tool call in JSON form, so the model learns the exact text pattern it must generate at test time. This is important because small differences in formatting can break the parser. For SpikeGPT, the processed text is then converted into the binary indexed dataset format used by the original training code. For GPT-2 baselines, the same logical examples are tokenized with the GPT-2 tokenizer.

The main SpikeGPT model uses the 216M-parameter checkpoint as initialization in the fine-tuning setting. Training from scratch is also tested, but it is much less stable on this dataset. The reason is that tool calling requires both language ability and schema learning. A randomly initialized model must learn Vietnamese or English-like phrasing, JSON syntax, tool names, and argument copying at the same time. A pretrained checkpoint already has basic token transition knowledge, so fine-tuning can focus on the commercial management schema.

Table 3.2: Main implementation choices used in the experiments.

Component	Choice in this project
Backbone	SpikeGPT-216M based on the public implementation of the original paper.
Task format	Causal language modeling over chat-style prompts and JSON tool calls.
Context length	1,024 tokens for the main SpikeGPT training setup.
HeadQK dimension	256 in the improved SpikeGPT variants.
Training strategy	Both from-scratch training and fine-tuning from a pretrained checkpoint.
Main objective	Standard causal language modeling cross-entropy; copy-aware loss is tested only as an ablation.
Evaluation parser	Extracts the generated tool name and JSON arguments, then compares them with ground truth.

This setup also explains why training loss alone is not enough to judge the model. Loss measures next-token prediction over the whole sequence, where many tokens are easy formatting tokens such as braces, quotes, commas, and common tool names. A model can reduce loss by learning the JSON template while still failing to copy rare IDs. For that reason, final evaluation is always based on task metrics, especially Intent Accuracy and Args Exact Match,

rather than only on train or validation loss.

3.3.5 GPT-2 Small — Zero-Shot Baseline (124M)

GPT-2 Small [2] is used without any fine-tuning. While it generates grammatically coherent text, it predates instruction-following and Chain-of-Thought techniques, exhibiting high hallucination rates and poor structured output consistency on Tool Calling tasks.

3.3.6 Fine-Tuned GPT-2 Medium — SFT Reference (355M)

GPT-2 Medium (355M parameters) is fine-tuned via Supervised Fine-Tuning (SFT) on our domain-specific Tool Calling dataset. With more Transformer layers and higher embedding dimensionality than GPT-2 Small, it learns complex relationships between user intent and API call structure, and can be trained on mid-range GPUs (e.g., NVIDIA RTX 3060/4060).

3.4 Dataset Construction and Preprocessing

Since no existing Vietnamese-language Tool Calling dataset for commercial management was available, the research team constructed the entire corpus from scratch using a four-step human-AI collaborative pipeline.

3.4.1 Construction Pipeline

Step (a) — Tool Definition. The six domain tools (Section 3.2) were formally defined with typed parameter schemas, serving as the foundation for all subsequent data generation.

Step (b) — Gold Standard Creation. Domain experts manually authored a curated set of perfect conversation examples spanning all six tools, covering the full range of user phrasings—short commands, polite requests, and ambiguous queries—serving as the quality reference for synthetic generation.

Step (c) — LLM-Based Synthetic Expansion. To scale the dataset without prohibitive annotation costs, we employed Gemini Pro and GPT-4 as data-generation agents [3]. Given the gold-standard examples as seed templates, the LLMs synthesized thousands of diverse multi-turn JSON conversations simulating realistic variations in user language, intent phrasing, and argument values. Each sample adheres to the role-based schema below:

Listing 3.3: Role-based multi-turn conversation schema.

```

1 {
2   "id": "sample_001",
3   "description": "Inventory check",
4   "tool_used": [
5     "check_inventory"
6   ],

```

```

7   "messages": [
8     {
9       "role": "user",
10      "content": "How many units of P100 are left?"
11    },
12    {
13      "role": "assistant",
14      "content": null,
15      "tool_calls": [
16        {
17          "name": "check_inventory",
18          "arguments": {
19            "product_id": "P100"
20          }
21        }
22      ]
23    },
24    {
25      "role": "tool",
26      "content": "{\"stock\": 50, \"product_id\": \"P100\"}"
27    },
28    {
29      "role": "assistant",
30      "content": "There are 50 units of P100."
31    }
32  ]
33 }

```

Step (d) — Data Cleaning. All synthetically generated samples underwent manual review and automated filtering to remove hallucinated tool names, malformed JSON, and semantically inconsistent argument values.

3.4.2 Dataset Scale and Splits

The raw corpus comprises 1,910 gold-quality samples covering all six tool categories. Following cleaning and augmentation (described below), the final training corpus is summarized in Table 3.3.

Table 3.3: Dataset split sizes after synthetic augmentation.

Split Partition	Samples	Intended Purpose
Training Set	56,000	Supervised model fine-tuning
Validation Set	7,600	Hyperparameter selection
Test Set	5,000	Final held-out evaluation
Total	68,600	

Note: The data augmentation process expanded the initial 1,910 gold-standard human annotations by approximately 35× to ensure robust coverage of phrasing and parameter variants.

Table 3.4: Representative samples from the gold corpus mapping intent to expected structured outputs.

ID	User Query (Example)	Target Tool	Expected JSON Arguments
smpl_01	“How many units of P100 are left in stock?”	check_inventory	{"product_id": "P100"}
smpl_02	“Create a new order for 5 units of P100 to 123 Main St.”	create_order	{"product_id": "P100", "qty": 5, "addr": "123 Main St."}
smpl_03	“What is the revenue from 2026-01-01 to 2026-01-31?”	revenue_analysis	{"start_date": "2026-01-01", "end_date": "2026-01-31"}
smpl_05	“Update order ORD-99 to have 10 units instead.”	update_order	{"order_id": "ORD-99", "qty": 10}
smpl_09	“Cancel my last order ORD-99 immediately.”	delete_order	{"order_id": "ORD-99"}

Note: The user queries demonstrate a mix of conversational, imperative, and interrogative phrasing styles naturally found in real-world commercial management scenarios. JSON structures are simplified for clarity.

3.4.3 Quality Assurance

We implemented a two-tier quality control pipeline to guarantee correctness.

Tier 1 — Manual Inter-Annotator Agreement (IAA). Team members independently reviewed randomly sampled batches of synthetic conversations. Disagreements on tool label correctness, argument completeness, or semantic consistency were resolved by consensus, and flagged samples were corrected or discarded.

Tier 2 — Automated Validation Pipeline. Every JSON file was subjected to eight automated checks, all of which must pass at 100% before the sample is accepted:

1. **Schema integrity:** Required top-level fields present and non-null.
2. **User message present:** At least one role: user turn exists.
3. **Tool call present:** At least one non-empty `tool_calls` array.
4. **Tool match:** `tool_used` metadata matches invoked tool names.
5. **Known tool:** All invoked tool names belong to the six-tool vocabulary.
6. **Args complete:** All required parameters are present and non-null.
7. **Args valid:** Parameter values are of the correct data type.
8. **Date valid:** Date parameters conform to YYYY-MM-DD and `start_date` $\leq$ `end_date`.

3.4.4 Preprocessing and Data Augmentation

- **Length filtering:** Conversations exceeding 600 tokens are discarded to fit within the 1,024-token context window of small Transformer models.
- **Tokenization:** Model-specific tokenizers (GPT-2 BPE; SpikeGPT character-level) are applied for each training pipeline.
- **Data augmentation** expands the 1,910 base samples to 68,600 total via three strategies: (i) *question paraphrasing* (synonym substitution and sentence reordering); (ii) *argument value perturbation* (randomizing product IDs, order IDs, and date ranges); (iii) *multi-intent chaining* (concatenating two single-turn dialogues into a two-step conversation).

3.4.5 Binary Spike Encoding for SpikeGPT

Unlike Transformers that operate on continuous embedding vectors, SpikeGPT requires input as discrete temporal spike streams. An additional binary encoding step maps each token index to a fixed-length binary vector, producing a spike activation sequence over simulated time steps. For a vocabulary of size V and a time-step window T , token t_i is encoded as:

$$\mathbf{s}_i \in \{0, 1\}^T, \quad \text{with } \|\mathbf{s}_i\|_1 \propto \text{rank}(t_i).$$

This rate-coding scheme preserves relative token frequency information while remaining compatible with the binary spike arithmetic of SpikeGPT’s RWKV backbone.

3.5 Data Pipeline Summary

Table 3.5 summarizes the end-to-end multi-turn data pipeline.

Table 3.5: Agentic Tool Calling conversational data pipeline.

Role	Content / Format	Functional Purpose
user	Natural language query	Captures raw user intent.
assistant	content: null	Internal tool trigger signal.
tool_calls	JSON: name & args	Extracts API routing parameters.
tool	Raw API response string	Injects database result back.
assistant	Final formatted reply	User-facing language synthesis.

Note: A complete transaction strictly follows this five-step schema to ensure seamless integration with the cloud execution engine.

3.6 Dataset Validity Analysis

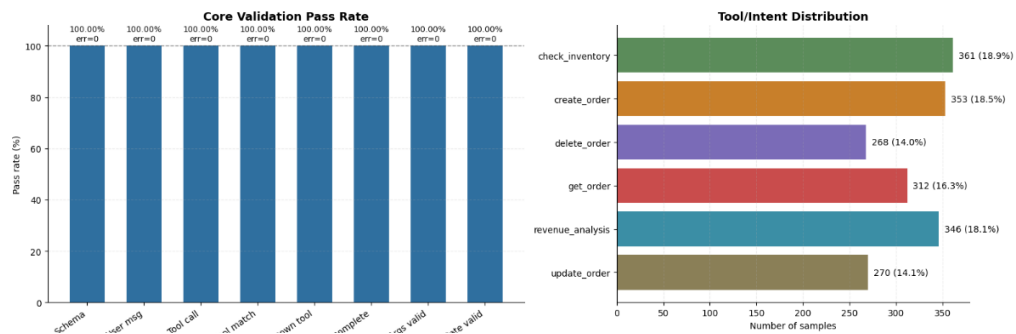


Figure 3.3: Dataset validity analysis showing statistical distribution.

All 1,910 base samples passed every automated validation check at a 100% pass rate with zero errors across all eight criteria. This perfect validation score confirms that the dataset is structurally sound, semantically consistent, and safe to use for fine-tuning without further manual intervention.

3.6.1 Tool and Intent Distribution

Table 3.6 reports the per-tool sample counts across the 1,910-sample corpus. The distribution is deliberately kept near-uniform to prevent label bias toward high-frequency tools.

Table 3.6: Dataset category distribution across the 1,910-sample corpus.

Category	Tool Routing Target	Samples	Share
Query	check_inventory	361	18.9%
Query	revenue_analysis	346	18.1%
Query	get_order	312	16.3%
Mutation	create_order	353	18.5%
Mutation	update_order	270	14.1%
Mutation	delete_order	268	14.0%
Total		1,910	100%

Note: The distribution is strictly maintained at a balanced ratio ($\approx 1.35\times$ max variance) to prevent the model from developing predictive bias toward frequently used APIs.

The Query group accounts for 53.3% of samples and the Mutation group for 46.7%. Within each group, no single tool dominates: the maximum imbalance ratio is $361/268 \approx 1.35\times$, well within acceptable bounds for fine-tuning small language models. This balanced distribution ensures fair evaluation across all six tool categories (Section 4).

Chapter 4

Experiments

4.1 Experimental Setup

We compare three model families on a held-out test set:

1. **GPT-2 Small** (124M): Zero-shot NLP baseline.
2. **Fine-tuned GPT-2 Medium** (355M): SFT upper-bound reference.
3. **SpikeGPT variants**: Multiple configurations to isolate architectural and training effects.

4.2 Evaluation Metrics

- **Valid JSON (%)**: Proportion of outputs that parse as syntactically valid JSON.
- **Intent Accuracy (%)**: Proportion of outputs with the correct tool name selected.
- **Args Exact Match (%)**: Proportion of outputs where all argument key-value pairs exactly match the ground truth.
- **Precision / Recall / F1**: Token-level intent classification metrics.

The metrics are intentionally separated because tool calling can fail in different ways. A response may be valid JSON but call the wrong tool. It may choose the correct tool but copy the wrong `order_id`. It may also copy some arguments correctly but miss one required field. A single score would hide these differences.

Intent Accuracy is computed as:

$$\text{Acc}_{\text{intent}} = \frac{\#\{\hat{y}_{\text{tool}} = y_{\text{tool}}\}}{N}.$$

It is easy to interpret: among all N test examples, it measures how often the predicted tool name matches the ground truth tool name.

Precision, Recall, and F1 are computed over the tool classes. For a given class,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

and

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

In the final table, we report weighted averages to account for small differences in class support. Intent Accuracy tells us the overall proportion of correct tool choices, while F1 also

reflects how errors are distributed across tools. This matters when one model performs well on common tools but poorly on rarer ones.

Args Exact Match is stricter:

$$\text{ArgsEM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{a}_i = a_i],$$

where a_i is the complete ground-truth argument dictionary. If one character in an identifier is wrong, the whole example is counted as incorrect. This strict metric is appropriate because real database operations can fail or affect the wrong record when arguments are almost correct but not exact.

4.3 Main Results

Table 4.1 presents the complete experimental results across all model variants and configurations.

Table 4.1: Comprehensive performance benchmark across model architectures.

Architecture / Model Variant	Valid JSON	Intent Acc.	Exact Match	Precision	Recall	F1-Score
SpikeGPT NoHeadQK (ep. 78)	<u>100.00%</u>	86.62%	0.00%	0.8781	0.8662	0.8659
SpikeGPT Scratch (ep. 78)	95.98%	84.64%	1.78%	0.8944	0.8464	0.8655
SpikeGPT Scratch (ep. 220)	91.52%	81.24%	4.68%	0.9022	0.8124	0.8463
SpikeGPT Finetune (ep. 220)	<u>99.80%</u>	<u>94.10%</u>	<u>13.94%</u>	<u>0.9459</u>	<u>0.9410</u>	<u>0.9415</u>
GPT-2 Small (zero-shot)	99.98%	96.52%	68.30%	0.9671	0.9652	0.9648
GPT-2 Medium (fine-tuned)	100.00%	96.78%	67.76%	0.9696	0.9678	0.9678

Note: Performance evaluated on a held-out test set of 5,000 conversational samples.

– Underline denotes the best performing variant among Spiking Neural Network (SNN) configurations.

^B Bold text indicates the highest score achieved overall across all architectures. Exact Match refers strictly to *Args Exact Match*.

4.4 Analysis and Failure Modes

4.4.1 JSON Validity and Intent Accuracy

Both the Transformer family and SpikeGPT Finetune demonstrate near-perfect ability to learn multi-turn dialogue structure, achieving valid JSON rates of 99.80%–100.00%. SpikeGPT

Finetune reaches 94.10% Intent Accuracy, closely approaching the GPT-2 family’s >96.50%. This confirms that SNN architectures can successfully learn the structural grammar of Tool Calling workflows when given sufficient fine-tuning.

4.4.2 Args Exact Match — The Critical Bottleneck

The most pronounced failure mode across all SpikeGPT variants is argument extraction. The NoHeadQK variant achieves 0.00% Exact Match, and even the best SpikeGPT Fine-tune reaches only 13.94%, compared to $\approx 68\%$ for GPT-2 variants. We attribute this gap to two fundamental properties of SNN-based processing:

1. **Binary spike encoding** compresses rich token information into sparse binary streams, causing information loss at the character/token level needed for precise entity extraction.
2. **Gradient convergence difficulty:** Non-differentiable spike activation functions complicate backpropagation, leading to slow or incomplete learning of fine-grained token-level patterns.

4.4.3 Hallucination in GPT-2 Small

Despite strong aggregate scores, GPT-2 Small (zero-shot) frequently produces hallucinated parameters or incorrect tool selections when confronted with multi-step logic. Fine-tuning GPT-2 Medium on the domain dataset effectively resolves this, achieving a favorable balance between performance and operational cost.

4.4.4 SpikeGPT Ablation Insights

Comparing SpikeGPT variants reveals three notable trends:

- **NoHeadQK vs. Scratch (ep. 78):** Removing HeadQK yields perfect JSON validity (100% vs. 95.98%) but zero argument extraction, suggesting HeadQK contributes to entity-level disambiguation.
- **Scratch ep. 78 vs. ep. 220:** Extended training slightly improves Args Exact Match (1.78% $\rightarrow$ 4.68%) but degrades JSON validity (95.98% $\rightarrow$ 91.52%), indicating overfitting to argument patterns at the cost of structural formatting.
- **Finetune (ep. 220):** Starting from a pre-trained checkpoint dramatically improves all metrics, reinforcing the importance of pre-training for SNN-based language models.

4.5 Additional Ablation Studies

4.5.1 Copy-Aware Loss

Because Args Exact Match is the weakest SpikeGPT metric, we also test whether changing the loss function can help the model focus more on copy-sensitive tokens. The copy-aware loss keeps causal language modeling unchanged, but gives a larger weight to target tokens

that already appear in the recent context. This is meant to help fields such as `order_id`, `product_id`, quantities, dates, and parts of addresses.

Table 4.2: Effect of copy-aware loss on the fine-tuned HeadQK setting.

Model setting	Valid JSON	Intent Acc.	Args EM	F1-score
Fine-tuned HeadQK, standard loss	99.80%	94.10%	13.94%	0.9415
Fine-tuned HeadQK, copy-aware loss	99.94%	91.28%	16.88%	0.9131

The result is mixed. Args Exact Match increases by 2.94 percentage points, which supports the idea that copy-sensitive tokens deserve special attention. However, Intent Accuracy and F1-score decrease. This means the model becomes slightly better at copying arguments but worse at choosing tools consistently. For that reason, the copy-aware loss is not selected as the final model. It is still useful as evidence that the team targeted the right weakness and measured the trade-off carefully.

4.5.2 ID Format Stress Test

Another possible explanation for low Args Exact Match is that the test set may contain identifier formats that are harder than those seen during training. For example, multi-segment IDs such as `ORD-DEMO-223` may be harder to copy than shorter IDs with a simple letter-number pattern. To check this, we created a more in-distribution test setting where ID formats are closer to the training data.

The result did not change much. The fine-tuned HeadQK model still produced high JSON validity and strong intent accuracy, but Args Exact Match remained low. This suggests that the main limitation is not only the surface format of IDs. The harder problem is copy-binding: the model must bind a string from the prompt to the correct JSON field and reproduce it without mutation.

4.5.3 Why Valid JSON Is Not Enough

The experiments also show why reporting only JSON validity would be misleading. SpikeGPT can produce parseable JSON almost all the time, but this does not guarantee that the call is useful. A tool-calling system needs all three levels to be correct: the syntax must be valid, the selected tool must match the user’s request, and every argument must match the required value. In commercial systems, an almost-correct identifier can still retrieve the wrong order or update the wrong record.

Chapter 5

Discussion

5.1 Viability of SNNs for Agentic AI

Our results confirm that SpikeGPT is a *viable but not yet competitive* architecture for Agentic AI. The model successfully learns conversation structure, intent classification, and tool routing—tasks requiring understanding of multi-turn dialogue patterns. This is a significant result, as prior SpikeGPT evaluations have focused exclusively on standard language modeling perplexity benchmarks.

5.2 The Encoding-Precision Trade-off

The fundamental challenge exposed by this study is the encoding-precision trade-off. Binary spike encoding enables dramatic energy efficiency but at the cost of token-level precision required for argument extraction. Future work should explore:

- Rate coding or temporal coding schemes that preserve more information per spike;
- Hybrid architectures that use continuous activations only for the argument extraction head;
- Specialized training objectives (e.g., contrastive loss on entity spans) that incentivize precise entity localization.

5.3 Deployment Considerations

For edge AI, robotics, and neuromorphic hardware deployments where energy efficiency is paramount and argument extraction precision requirements are relaxed, the current SpikeGPT Finetune variant already provides meaningful utility. Its 94.10% Intent Accuracy means the correct tool is selected in over 9 out of 10 queries—sufficient for many real-world commercial routing applications.

5.4 Argument Extraction as a Copy-Binding Problem

Argument extraction is harder than intent recognition because it requires two actions at the same time. First, the model must understand the semantic role of a span in the user prompt. Second, it must reproduce the span exactly in the right JSON field. For example, when a user asks to cancel order ORD-99, the model must understand that the tool is `delete_order` and that ORD-99 must be placed under `order_id`. If the model outputs ORD-98, the intent is still correct but the real operation is wrong.

Transformer models have a natural advantage here because attention can directly link the current output position to earlier input tokens. SpikeGPT, by contrast, relies on recurrent state compression and spike-based representations. This design is attractive for efficient computation, but it can lose small surface details that matter for copying. The experimental gap in Args Exact Match is therefore consistent with the architecture, not just a random training issue.

5.5 Lessons from Failed Improvements

Several attempted improvements did not become the final model, but they are still important for interpreting the project. Adding reasoning-style text before the JSON output made the sequence longer and introduced more opportunities for repetition and formatting errors. Combining long reasoning text with HeadQK also made training less stable, because the model had to generate free-form language and strict JSON in the same output.

The copy-aware loss was more controlled. It improved Args Exact Match, but it also reduced Intent Accuracy and F1-score. This shows that the model has limited capacity to optimize all parts of tool calling at once. Giving more weight to copied tokens can pull learning away from tool selection. For the final system, we therefore keep the standard loss and treat copy-aware loss as an ablation result.

5.6 Practical System Design

A practical deployment should not allow the current SpikeGPT model to execute all tools without checks. The safest design is a hybrid pipeline. SpikeGPT can serve as a low-power first-stage router because its intent accuracy is high. A deterministic validator can then check whether required arguments are present, whether IDs appear in the user prompt or database, and whether values have the correct type.

For read-only actions such as inventory checking or order lookup, the system may allow automatic execution when confidence is high. For risky actions such as updating or deleting an order, the system should ask for confirmation if the extracted arguments are uncertain. This design uses SpikeGPT where it is strongest while protecting the system from the exact-copying weakness observed in the experiments.

5.7 Future Improvements

The most direct future direction is constrained decoding. If the model predicts that a field should contain an `order_id`, the decoder can restrict candidates to identifiers found in the input prompt or retrieved from the database. This would reduce hallucinated IDs without changing the full SpikeGPT backbone.

A second direction is a pointer or copy module. Instead of generating every argument token from the whole vocabulary, the model can explicitly copy spans from the prompt. This is

closer to how many sequence-to-sequence systems handle names and rare tokens. Such a module would directly target the failure mode measured by Args Exact Match.

A third direction is field-level training and evaluation. The current Args Exact Match metric is intentionally strict, but it does not reveal whether the model is worse at IDs, dates, quantities, or addresses. Reporting field-level accuracy would make the next improvement more focused and easier to justify.

Chapter 6

Conclusion

We presented the first systematic evaluation of SpikeGPT as an Agentic AI for Tool Calling in commercial management systems. Our key contributions are:

1. A domain-specific multi-turn Tool Calling dataset for commercial management, combining human-curated gold standards with LLM-synthesized samples.
2. An end-to-end evaluation framework measuring Valid JSON rate, Intent Accuracy, and Args Exact Match across SNN and Transformer architectures.
3. Empirical demonstration that SpikeGPT Finetune achieves 99.80% JSON validity and 94.10% Intent Accuracy, establishing SNNs as structurally capable Agentic learners.
4. Identification of Args Exact Match (13.94% vs. $\approx 68\%$ for GPT-2) as the primary bottleneck, linked to binary spike encoding and gradient convergence challenges.

SpikeGPT demonstrates substantial promise as an energy-efficient alternative for Agentic AI, particularly for edge deployment scenarios. Bridging the argument extraction gap through improved encoding mechanisms represents the most critical direction for future research.

Bibliography

- [1] Zhu, R.-J., Zhao, Q., Li, G., & Eshraghian, J.K. (2023). SpikeGPT: Generative Pre-trained Language Model with Spiking Neural Networks. *arXiv preprint arXiv:2302.13939*.
- [2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Blog*, 1(8), 9.
- [3] Luo, Z., et al. (2025). Self-Training Large Language Models for Tool-Use Without Demonstrations. *arXiv preprint*.
- [4] Maass, W. (1997). Networks of Spiking Neurons: The Third Generation of Neural Network Models. *Neural Networks*, 10(9), 1659–1671.
- [5] Peng, B., et al. (2023). RWKV: Reinventing RNNs for the Transformer Era. *arXiv preprint arXiv:2305.13048*.
- [6] Brown, T.B., et al. (2020). Language Models are Few-Shot Learners. *Advances in NeurIPS*, 33, 1877–1901.
- [7] Vaswani, A., et al. (2017). Attention is All You Need. *Advances in NeurIPS*, 30.